🧠 Dynamic Memory Management Functions

📖 Definition

Dynamic memory allocation lets a program request memory from the **heap** while it is actually running, instead of everything being fixed at compile time — and release it once it's no longer needed. C provides four standard functions for this, all declared in `<stdlib.h>`.

❓ Why Do We Need It?

An array like `int arr[10];` has a size fixed forever at compile time — too small, and you run out of room; too big, and you waste memory. Dynamic allocation lets the **size be decided while the program runs** — e.g. based on user input — and lets you request more memory later, or give memory back the moment it's done being used.

1️⃣ malloc() — Memory Allocation

```
ptr = (cast_type*) malloc(size_in_bytes);
```

Allocates a single block of the requested size — but does **not** initialize it, so it holds garbage values until you set them yourself.

2️⃣ calloc() — Contiguous Allocation

```
ptr = (cast_type*) calloc(n, size_of_each_element);
```

Allocates memory for `n` elements and — unlike `malloc()` — initializes **every byte to zero** automatically.

3️⃣ realloc() — Re-allocation

```
ptr = realloc(ptr, new_size);
```

Resizes a block that was previously allocated (grow or shrink it), preserving the existing data. The block may be **moved** to a new address if it can't be resized in place — always re-assign the result back to `ptr`.

4️⃣ free() — Deallocation

```
free(ptr);
```

Releases previously allocated memory back to the system. Forgetting this causes a **memory leak**. Right after freeing, it's good practice to set the pointer to `NULL` — otherwise it becomes a **dangling pointer** (remember that one, from the Pointers unit?).

💻 Example Program — All Four Functions Together

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*) malloc(3 * sizeof(int));      // 3 ints, garbage values
    for (int i = 0; i < 3; i++) arr[i] = i + 1;

    int *zeros = (int*) calloc(3, sizeof(int));      // 3 ints, all zero
    printf("calloc value at index 0 = %d\n", zeros[0]);

    arr = (int*) realloc(arr, 5 * sizeof(int));      // grow from 3 to 5 ints
    arr[3] = 4; arr[4] = 5;

    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    printf("\n");

    free(arr); free(zeros);                          // give memory back
    arr = NULL; zeros = NULL;                         // avoid dangling pointers
    return 0;
}
```

▶ Output

```
calloc value at index 0 = 0
1 2 3 4 5
```

Function	Initializes Memory?	Typical Use
`malloc()`	✗ No (garbage)	Fast allocation when you'll fill every value yourself
`calloc()`	✓ Yes (all zero)	When a clean, zeroed-out block is needed
`realloc()`	Keeps old data	Growing/shrinking a block you already allocated
`free()`	—	Releasing memory once you're done with it

⚠ Always check that the returned pointer isn't `NULL` before using it — allocation can fail if the system is out of memory. And never touch a pointer after `free()`-ing it without reassigning it first.

✨ Fun Tip

malloc = renting an empty warehouse (still has the last tenant's junk) 🏚. calloc = renting one that's been cleaned ✨. realloc = asking the landlord to resize your space. free = handing back the keys 🔑!

🔤 String Handling Functions — Part 1

📖 Definition

C provides a set of ready-made functions in `<string.h>` so you don't have to write pointer-walking loops by hand every time you need to measure, compare, or copy a string (although, as you saw in the Pointers unit, that's exactly how they work internally). This page covers the first three; Part 2 covers the rest.

1️⃣ strlen() — String Length

```
size_t strlen(const char *str);
```

Returns the number of characters before `'\0'` — the null terminator itself is **not** counted.

```
char word[] = "Hello";
printf("%lu", strlen(word));   // 5
```

2️⃣ strcmp() — String Compare

```
int strcmp(const char *str1, const char *str2);
```

Compares two strings **character by character** (lexicographically, like a dictionary) and returns:

Return Value	Meaning
`0`	The two strings are exactly equal
Negative	`str1` comes before `str2` alphabetically
Positive	`str1` comes after `str2` alphabetically

3️⃣ strcpy() — String Copy

```
char *strcpy(char *dest, const char *src);
```

Copies `src` into `dest`, including the null terminator. `dest` must already have enough space allocated for the whole string.

⚠ `strcpy()` does **not** check whether `dest` is big enough — writing past its end is a classic cause of buffer overflows. When the destination size isn't guaranteed, prefer the safer `strncpy()` (covered in Part 2).

💻 Example Program — strlen, strcmp & strcpy Together

```
#include <stdio.h>
#include <string.h>

int main() {
    char name1[] = "Aman";
    char name2[] = "Aman";
    char copy[10];

    printf("Length of name1 = %lu\n", strlen(name1));

    if (strcmp(name1, name2) == 0)
        printf("name1 and name2 are the SAME\n");
    else
        printf("name1 and name2 are DIFFERENT\n");

    strcpy(copy, name1);              // copy name1 into 'copy'
    printf("Copied string = %s\n", copy);
    return 0;
}
```

▶ Output

```
Length of name1 = 4
name1 and name2 are the SAME
Copied string = Aman
```

📝 Note

- `strlen()` returns type `size_t` (an unsigned integer) — print it with `%lu`, not `%d`.
- `strcmp()` is **case-sensitive**: `"Hi"` and `"hi"` are considered different strings.

✨ Fun Tip

strlen = counting footsteps to the finish line 👣. strcmp = a dictionary race — who comes first? strcpy = photocopying one string onto a blank page 📋!

🔤 String Handling Functions — Part 2

📖 Recap

Part 1 covered `strlen()`, `strcmp()` and `strcpy()`. This page covers the remaining three commonly used functions from `<string.h>`.

4️⃣ strncpy() — Safer, Bounded Copy

```
char *strncpy(char *dest, const char *src, size_t n);
```

Copies at **most** `n` characters from `src` into `dest` — much safer than `strcpy()` when `n` is chosen to match `dest`'s real size.

> ⚠ If `src` is **longer than or equal to** `n`, the result may **not** be null-terminated. It's common practice to manually force it: `dest[n-1] = '\0';`

5️⃣ strcat() — String Concatenate

```
char *strcat(char *dest, const char *src);
```

Appends `src` onto the **end** of `dest` — it overwrites `dest`'s old null terminator and adds a fresh one after the combined text. `dest` must have enough spare room for both strings.

6️⃣ strstr() — Substring Search

```
char *strstr(const char *haystack, const char *needle);
```

Searches for the first occurrence of `needle` inside `haystack`. Returns a pointer to where the match **begins**, or `NULL` if it isn't found anywhere.

💻 Example Program — strncpy, strcat & strstr Together

```c
#include <stdio.h>
#include <string.h>

int main() {
    char dest[20];
    strncpy(dest, "Hello", 5);
    dest[5] = '\0';                      // force null-termination

    strcat(dest, ", World!");            // append onto dest
    printf("After strcat: %s\n", dest);

    char *pos = strstr(dest, "World");
    if (pos != NULL)
        printf("Found \"World\" starting at: %s\n", pos);
    else
        printf("Not found\n");
    return 0;
}
```

▶ Output

```
After strcat: Hello, World!
Found "World" starting at: World!
```

📋 Summary — All 6 Functions at a Glance

Function	Purpose	Returns
`strlen()`	Measures string length	Length (excluding `'\0'`)
`strcmp()`	Compares two strings	0 / negative / positive
`strcpy()`	Copies a whole string	Pointer to `dest`
`strncpy()`	Copies up to n characters (safer)	Pointer to `dest`
`strcat()`	Appends one string onto another	Pointer to `dest`
`strstr()`	Finds a substring inside a string	Pointer to match, or `NULL`

✨ Fun Tip

strncpy = photocopying with a page limit 📋. strcat = gluing one string onto the tail of another 💞. strstr = playing "Where's Waldo?" inside a sentence 🔍!

🗂️ Storage Classes — Types

📖 Definition

A **storage class** in C determines four things about a variable: **where** it is stored, its **default initial value**, its **scope** (where its name is visible), and its **lifetime** (how long it stays alive). C has four storage classes, chosen with a keyword placed before the data type in a declaration.

Storage Class	Keyword	Stored In	Default Value
Automatic	`auto`	Stack (RAM)	Garbage (unpredictable)
Register	`register`	CPU register (if available)	Garbage (unpredictable)
Static	`static`	Fixed memory (data segment)	0 (zero-initialized)
External	`extern`	Fixed memory (data segment)	0 (zero-initialized)

1️⃣ auto — The Default for Local Variables

Every local variable inside a function is `auto` by default — you almost never need to type the keyword yourself.

```
void func() {
    auto int x = 5;   // identical to just writing: int x = 5;
}
```

2️⃣ register — A Request for Speed

Asks the compiler to store the variable in a fast CPU register instead of ordinary RAM — useful for a variable accessed very frequently, like a tight loop counter. The compiler may ignore the request if no register is free, and you cannot take the address (`&`) of a `register` variable, since registers don't have memory addresses.

```
register int i;
for (i = 0; i < 1000; i++) { /* runs very frequently */ }
```

3️⃣ static — Remembers Its Value

A `static` local variable is initialized only **once**, and then keeps its value between function calls, even though it's still only visible inside that function. At global scope, `static` instead restricts a variable's visibility to just the file it's defined in.

4️⃣ extern — Shares a Variable Across Files

Declares that a variable is defined **elsewhere** (often another file), without creating a new copy of it — this lets multiple files share one single global variable safely.

```
// file1.c
int total = 0;              // DEFINITION — actually creates the variable

// file2.c
extern int total;           // DECLARATION — reuses file1's "total", no new copy
```

💻 Example Program — static in Action

```
#include <stdio.h>

void counter() {
    static int count = 0;   // initialized ONCE, ever
    count++;
    printf("Called %d time(s)\n", count);
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

▶ Output

```
Called 1 time(s)
Called 2 time(s)
Called 3 time(s)
```

📝 Note

- If `count` had been a plain (`auto`) local variable instead, it would reset to 0 on every call — the output would be `Called 1 time(s)` three times in a row.
- `register` is only a *request* — modern compilers are often better than the programmer at deciding this anyway, so it's used less often today.

✨ Fun Tip

auto = a hotel room (cleared out when you check out) 🏨. static = your own locker that keeps your stuff between visits 🔒. extern = a shared family locker other rooms can also access 👨‍👩‍👧!

🗺 Scope Rules, Declaration & Definition

🕮 Definition

Scope is the region of a program where a variable's name is recognized and can legally be used. Combined with storage class, it decides visibility rules that keep large programs organized and free of naming clashes.

🌐 Local (Block) Scope vs Global (File) Scope

Aspect	Local Variable	Global Variable
Declared	Inside a function or block `{ }`	Outside every function, at the top of the file
Default storage class	`auto`	`extern`-like (external by default)
Visible in	Only within its own block	The whole file, from its declaration onward (and other files, via `extern`)
Lifetime	Destroyed when the block ends	Exists for the entire program run

🔄 Declaration vs Definition — The Crucial Difference

Declaration	Definition
Just announces a name and type to the compiler	Actually creates the variable and allocates storage for it
`extern int total;`	`int total;` or `int total = 0;`
Can appear **many times** across files	Must happen **exactly once** in the whole program

This is exactly the same declaration-vs-definition idea you saw with **functions** — a prototype announces a function without creating it; a variable declared with `extern` announces it without creating it either.

💻 Example Program — Local Variable Shadowing a Global One

```c
#include <stdio.h>

int value = 100;              // global variable (file scope)

void show() {
    int value = 5;             // LOCAL variable — shadows the global one, only inside show()
    printf("Local value = %d\n", value);
}

int main() {
    printf("Global value (before) = %d\n", value);
    show();
    printf("Global value (after)  = %d\n", value);  // completely unaffected
    return 0;
}
```

▶ Output

```
Global value (before) = 100
Local value = 5
Global value (after)  = 100
```

⚠ Giving a local variable the **same name** as a global one doesn't cause an error — it just "shadows" (hides) the global inside that block. It works, but it's confusing to read, so it's best avoided by choosing distinct names.

✨ Fun Tip

A global variable is a town's shared notice board 📜 — everyone can read it; a local variable is a sticky note on your own desk 📝 that disappears once you tidy up!

Compiled & prepared by
Anand Sagar Rout
B.Sc. Computer Science | GMU